

10x slower to train [10]. Therefore, subsequent works focused on convolutional neural network based blocks, which are faster to train. However, recently there is renewed interest in efficient RNNs and state space models both of which aim to address the inefficiencies of traditional RNNs. These newer developments aim to combine the sequential processing capabilities of RNNs with improved computational and memory efficiency, making them viable for longer sequences and higher throughput requirements. Key advancements include: 1.) State Space Models (SSMs): Recent models like S4 (Structured State Space for Sequence Modeling) [16] and its variants propose replacing traditional recurrent operations with structured linear systems that can process sequences in parallel while retaining the ability to model long-range dependencies. By leveraging the mathematical properties of state space representations, these models reduce the need for sequential backpropagation, offering significant speed and memory improvements. These efforts culminated in the recently introduced S6 model [17], which uses a specific Mamba block around the SSM core, gaining considerable attention due to its strong performance on sequential tasks. 2.) Work on parallelization of RNNs: where [18] linearized and diagonalized the recurrence; whereas [19] built on improving the LSTM model; and [20] introduced a new minimal version of GRUs and LSTMs, called minLSTM and minGRU.

In this work, we analyze the minGRU model for its use in the Turbo autoencoder structure. For that we combine it with the Mamba block for SSMs and integrate it into the parallel Turbo structure of the encoder. We will show that it matches and at times outperforms the convolutional network performance, while being as fast to train as the convolutional networks for smaller sequence length, and having significant advantages at long block lengths.

a) Notation: In this work, we distinguish between random vectors in the communication model and deterministic vectors in the neural network: an n -dimensional random vector is denoted by an uppercase letter with a superscript, e.g. X^n , whose components are $X_1, X_2, \dots$, and whose realized value is x^n . In contrast, neural-network vectors are written in bold lowercase with a time subscript, e.g. $\mathbf{x}_t$, to represent the input at time t in a recurrent model. For example, X^n corresponds to a random codeword of length n transmitted over the channel (Section III), whereas $\mathbf{x}_t$ is the deterministic input signal at time t in the MinGRU-based block (Section II).

II. MINGRU-BASED BLOCK

A. Introduction to Recurrent Neural Networks and Gated Recurrent Units

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by maintaining a hidden state that captures information about previous inputs. Unlike feedforward neural networks, RNNs share parameters across time steps, enabling them to model temporal dependen-

cies effectively. Formally, given an input sequence $\{\mathbf{x}_t\}_{t=1}^T$, the hidden state $\mathbf{h}_t$ at time t is computed as:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \theta), \quad (1)$$

where f is a non-linear function (e.g., a combination of matrix multiplications and activation functions), $\mathbf{h}_{t-1}$ is the hidden state from the previous time step, $\mathbf{x}_t$ is the input at time t , and θ are the network parameters.

One of the key challenges in training RNNs is the vanishing/exploding gradient problem, which limits their ability to capture long-term dependencies. Gated architectures like the Gated Recurrent Unit (GRU) were introduced to address this limitation.

B. Gated Recurrent Unit (GRU)

The GRU is a simplified yet effective variant of the Long Short-Term Memory (LSTM) network. It uses gating mechanisms to control the flow of information, which helps in preserving long-term dependencies and mitigating the vanishing gradient issue. The GRU maintains a single hidden state $\mathbf{h}_t$, updated as follows:

- 1) **Update Gate:** The update gate $\mathbf{z}_t$ determines how much of the past hidden state $\mathbf{h}_{t-1}$ should be retained:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z),$$

where σ is the sigmoid activation function, and $\mathbf{W}_z, \mathbf{U}_z, \mathbf{b}_z$ are learnable parameters.

- 2) **Reset Gate:** The reset gate $\mathbf{r}_t$ controls how much of the past hidden state should influence the current computation:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r).$$

- 3) **Candidate Hidden State:** A candidate hidden state $\tilde{\mathbf{h}}_t$ is computed using the reset gate:

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h),$$

where $\odot$ denotes the element-wise product.

- 4) **Final Hidden State:** The final hidden state $\mathbf{h}_t$ is a convex combination of the previous hidden state and the candidate hidden state, weighted by the update gate:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.$$

Neglecting the bias terms, and streamlining the matrices as linear operations Linear_{d_h} with a certain matching hidden dimension d_h , a GRU cell has the following simplified form:

$$\begin{aligned} \mathbf{z}_t &= \sigma(\text{Linear}_{d_h}([\mathbf{x}_t, \mathbf{h}_{t-1}])) \\ \mathbf{r}_t &= \sigma(\text{Linear}_{d_h}([\mathbf{x}_t, \mathbf{h}_{t-1}])) \\ \tilde{\mathbf{h}}_t &= \tanh(\text{Linear}_{d_h}([\mathbf{x}_t, \mathbf{r}_t \odot \mathbf{h}_{t-1}])) \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned}$$